

How to AI an AI for Almost Anything

Lecture 2 - Context Evolution

Paul Liang

Assistant Professor

MIT Media Lab & MIT EECS



<https://pliang279.github.io>

ppliang@mit.edu

 [@pliang279](https://twitter.com/pliang279)



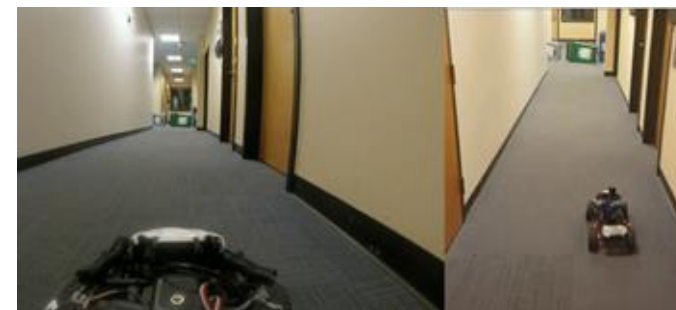
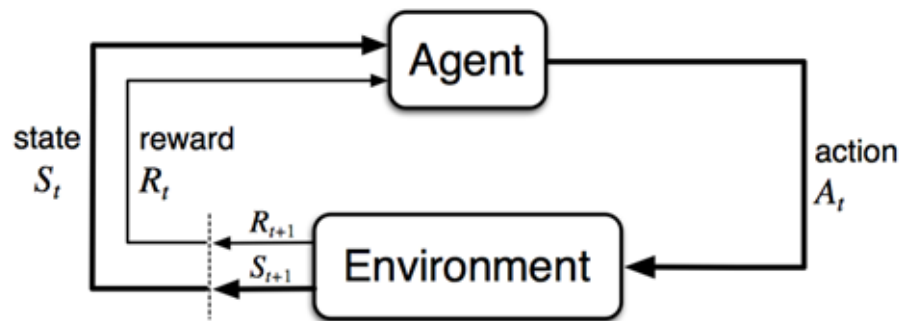
Announcements

- Everyone notified of for-credit or listener status
- Everyone added to canvas
- Project preferences form on canvas
- First reading assignment will be out tonight on context evolution

Today's Lecture

- Types of context: text and multimodal, prompts, memory
- Prompt optimization
- Long-context models
- Context management and memory
- Guest speaker: agentic context engineering
- Demo of context evolution

What is an Agent?



An agent perceives its environment through sensors and acts upon the environment through actuators.

Environment: code repo, website, application, real world.

States/observations: messages, files, webpages, tool results.

Actions: text, file edits, API calls, shell commands, physical movements.

Rewards: test cases, task completions, LLM judges, user feedback.

What is Context?

Context: Everything the model reads before generating the next token

types

developer

system prompt

instructions

tool definitions

examples

memory

user

user message

environment

files

search results

tool output

the model

its own thoughts and actions

uses

Inform

new information

Steer

instructions, role, format

Demonstrate

examples

Compute

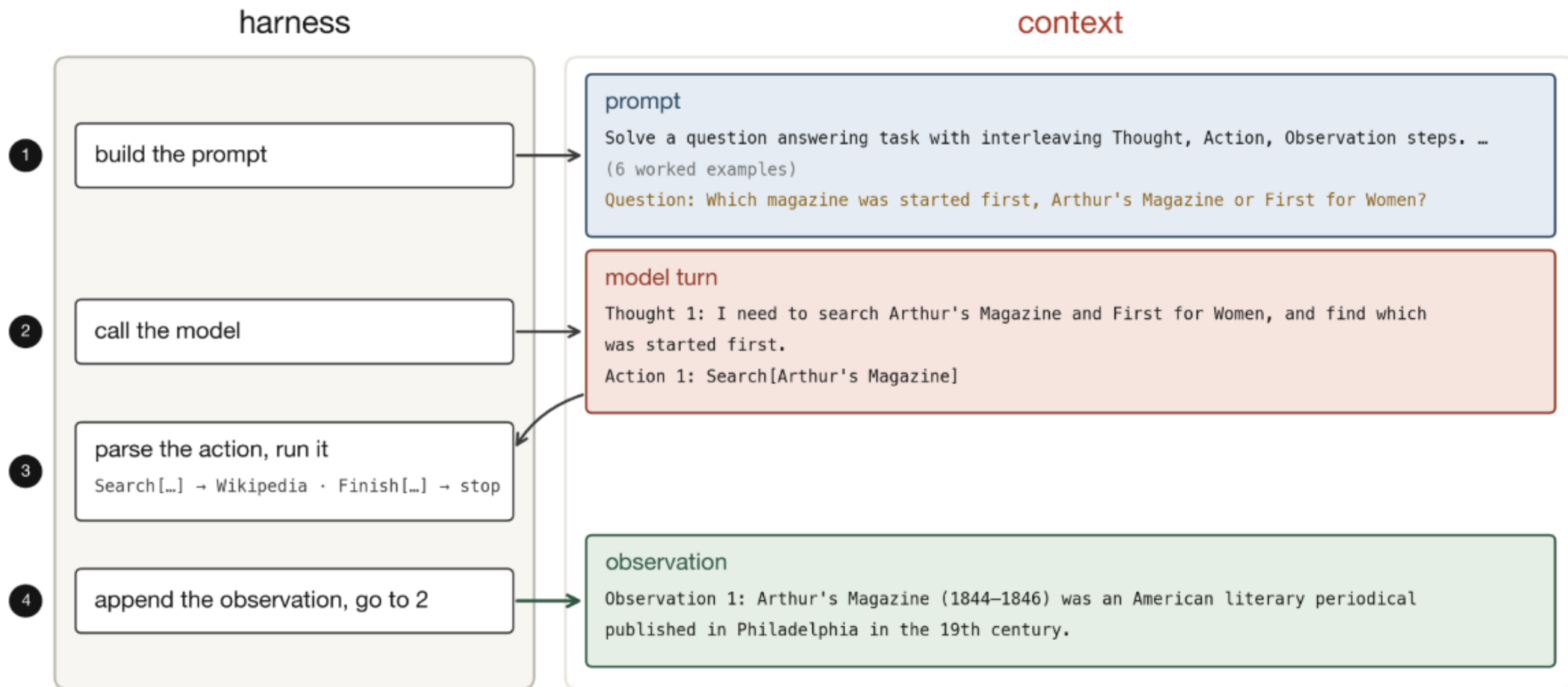
thinking step by step

Correct

feedback from the last attempt

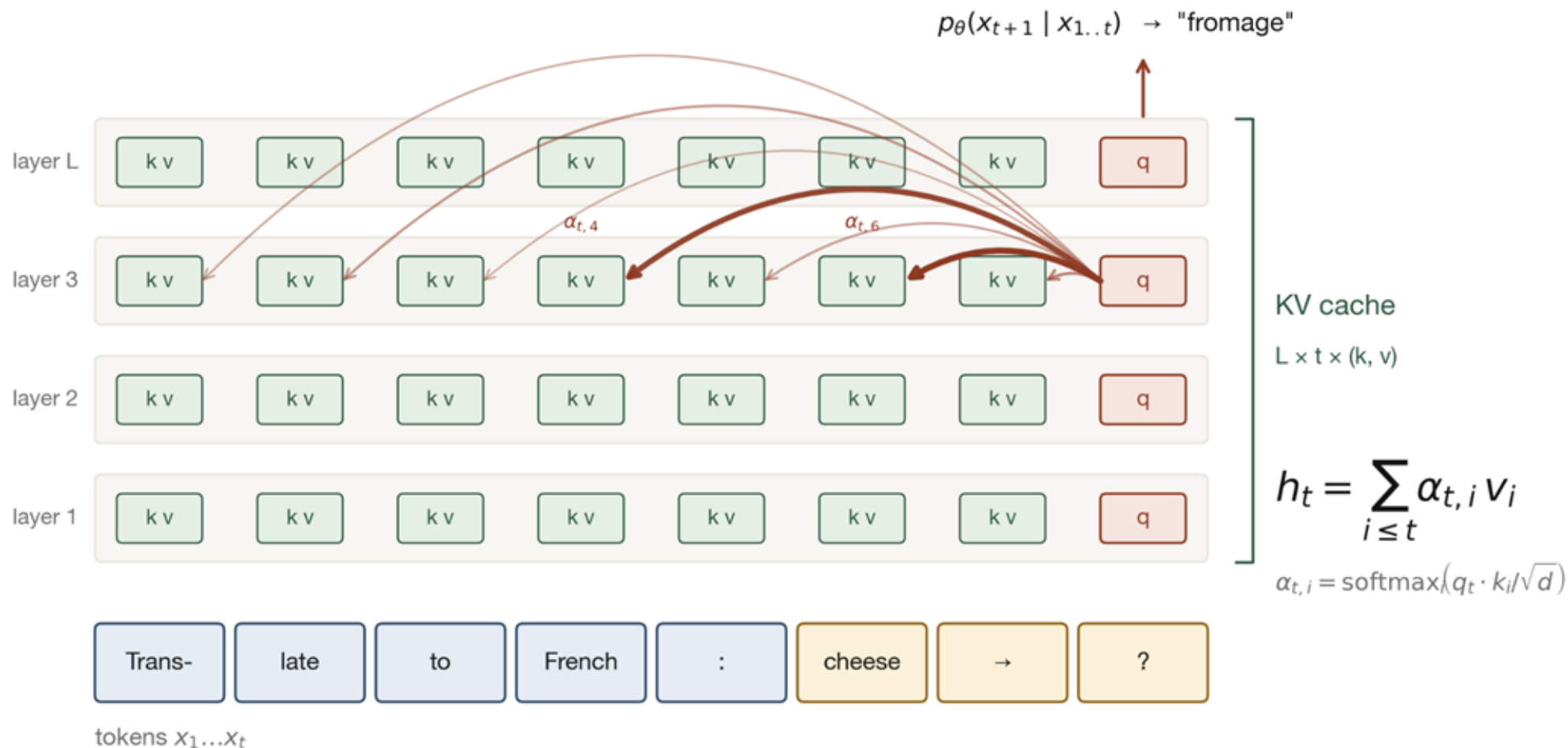
Context vs Harness?

Harness: Code around the model that manages its context and execution flow to complete tasks



How does Context Change Behavior?

When predicting the next token, the model attends to the preceding context



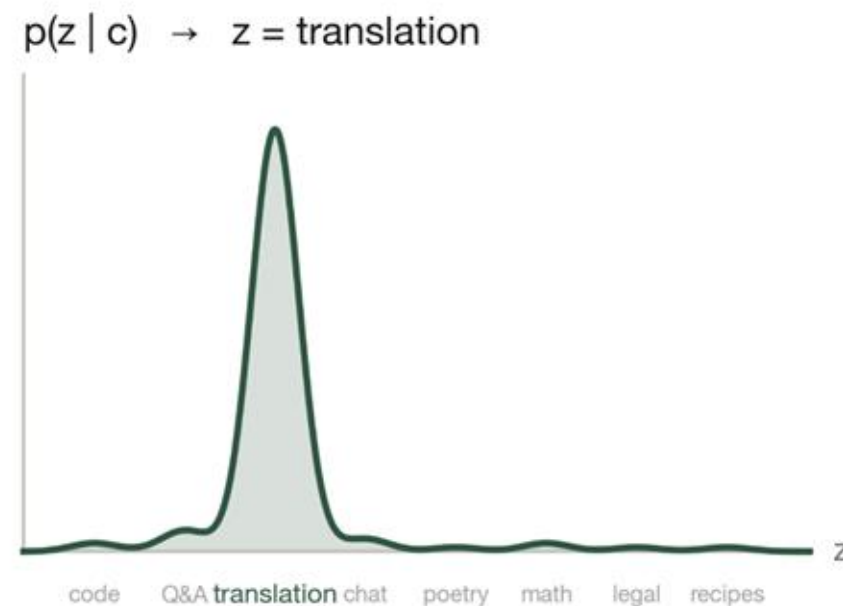
How does Context Change Behavior?

which task am I doing?

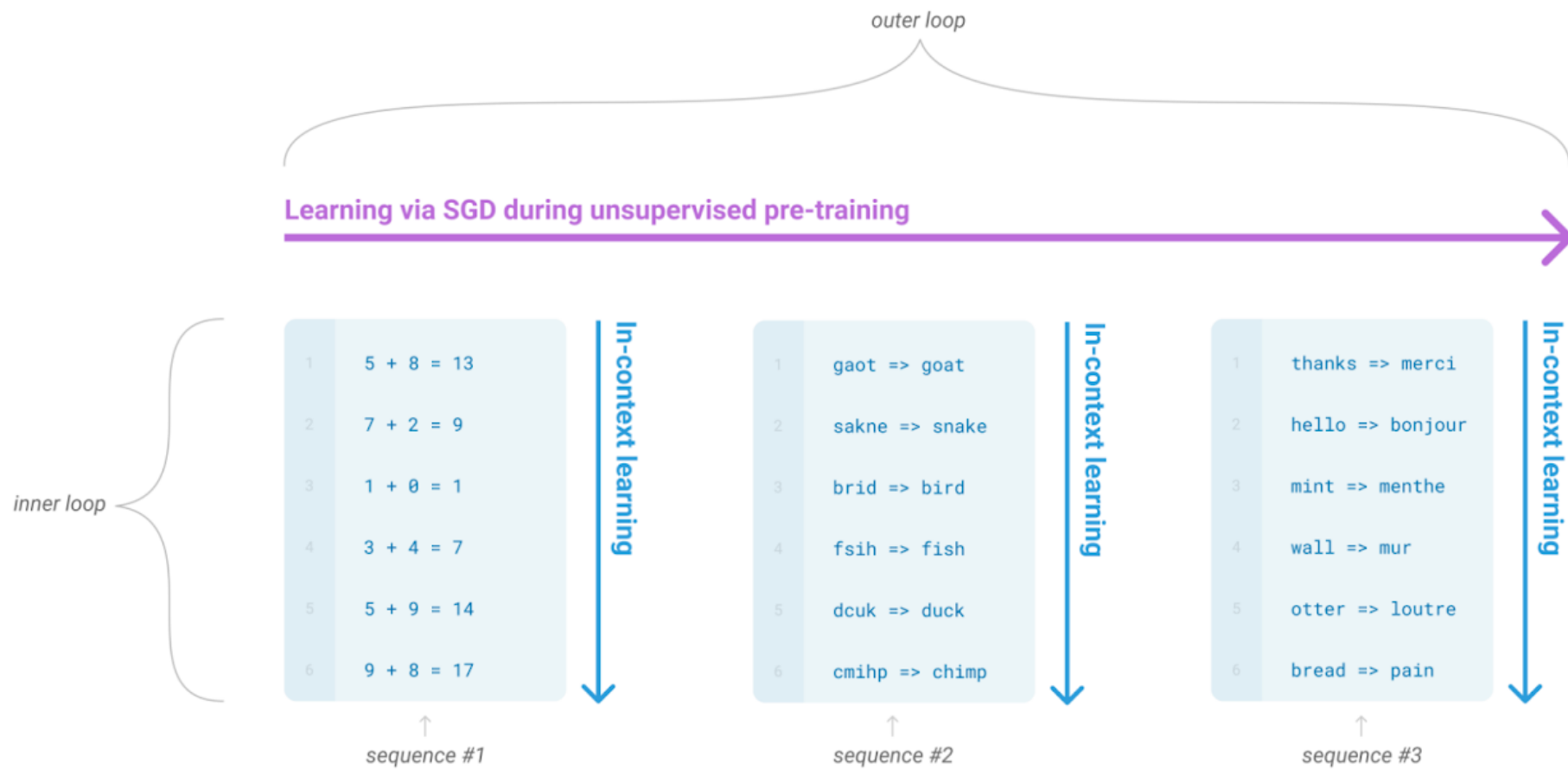


context c

sea otter → loutre de mer
peppermint → menthe poivrée
cheese → fromage



In-Context Learning



Chain of Thought Reasoning

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

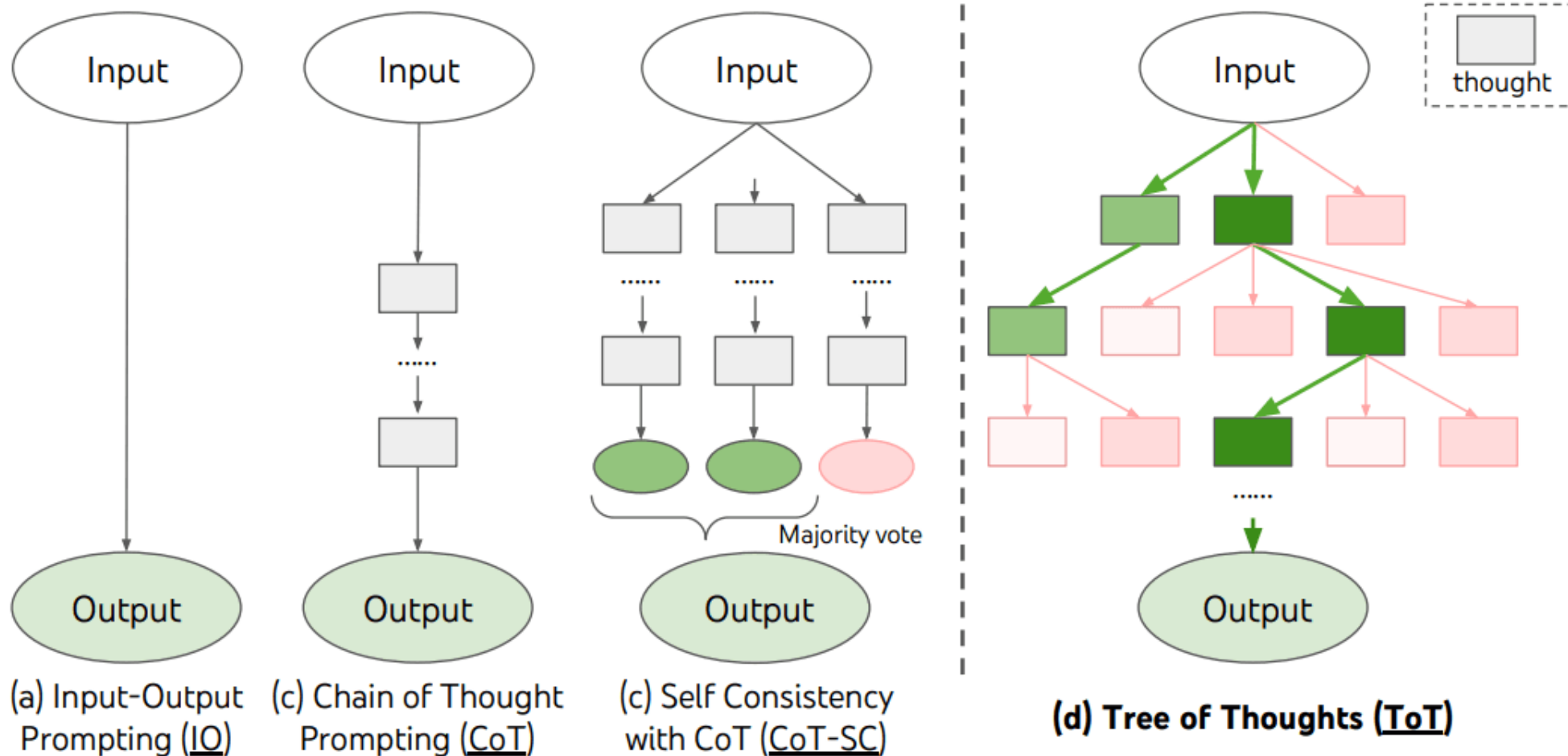
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

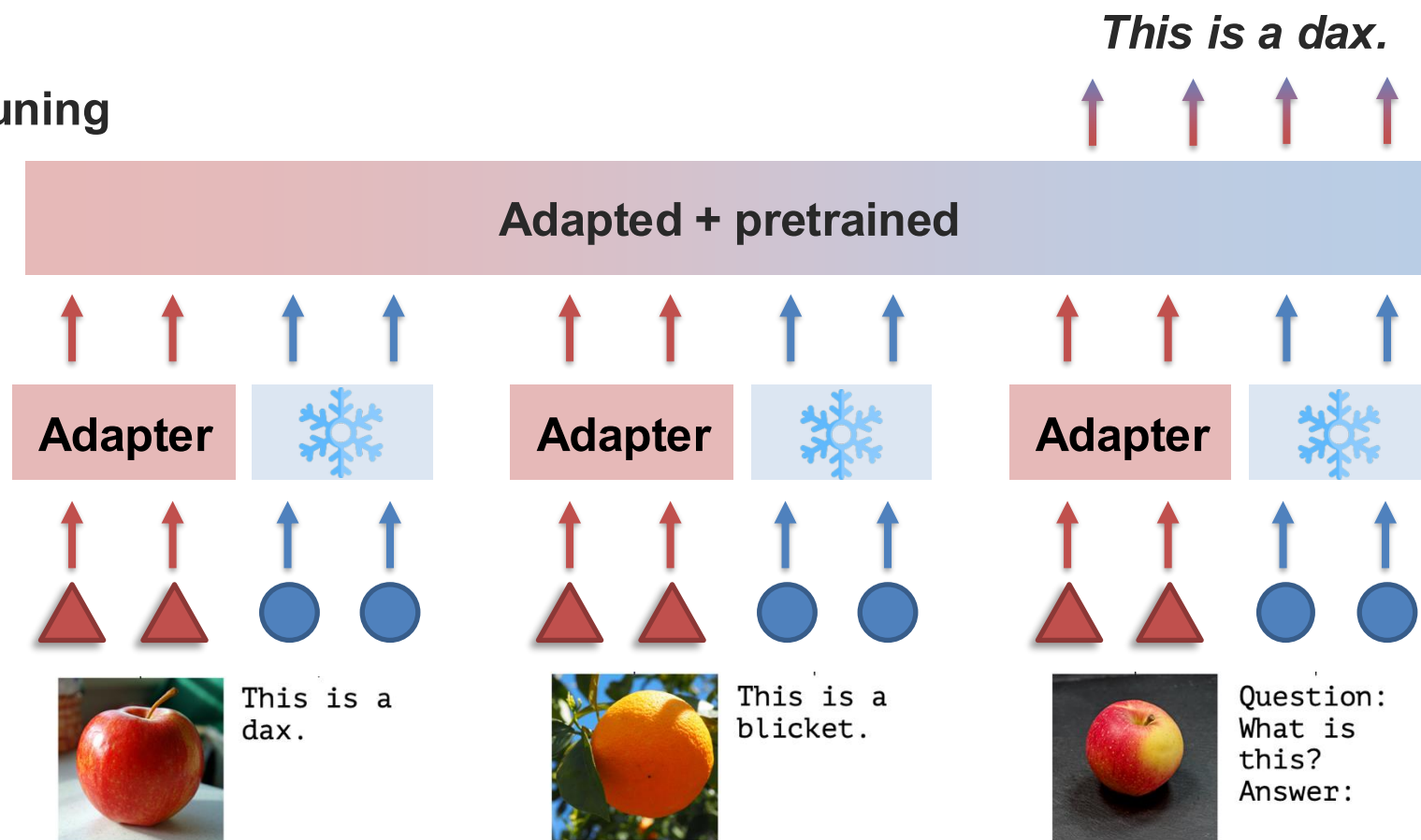
Tree of Thoughts Reasoning



Multimodal Context

Conditioning via prefix tuning

Modeling $p(\text{text} \mid \text{image})$:

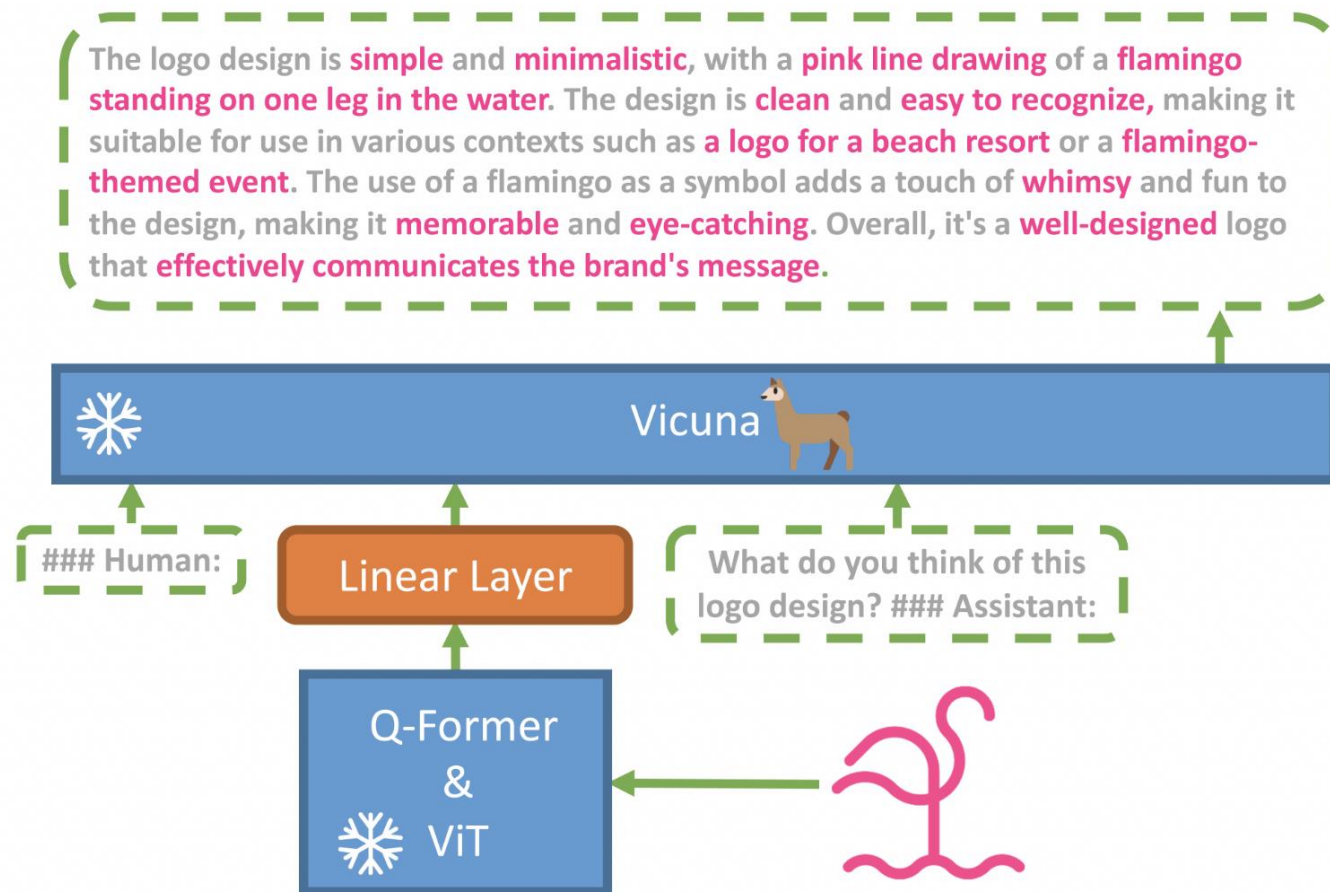


Multimodal Context

Mini-GPT4

Stage 1: **Alignment** using paired image-text data.

Stage 2: **Instruction tuning** using image + text instructions and example completions.

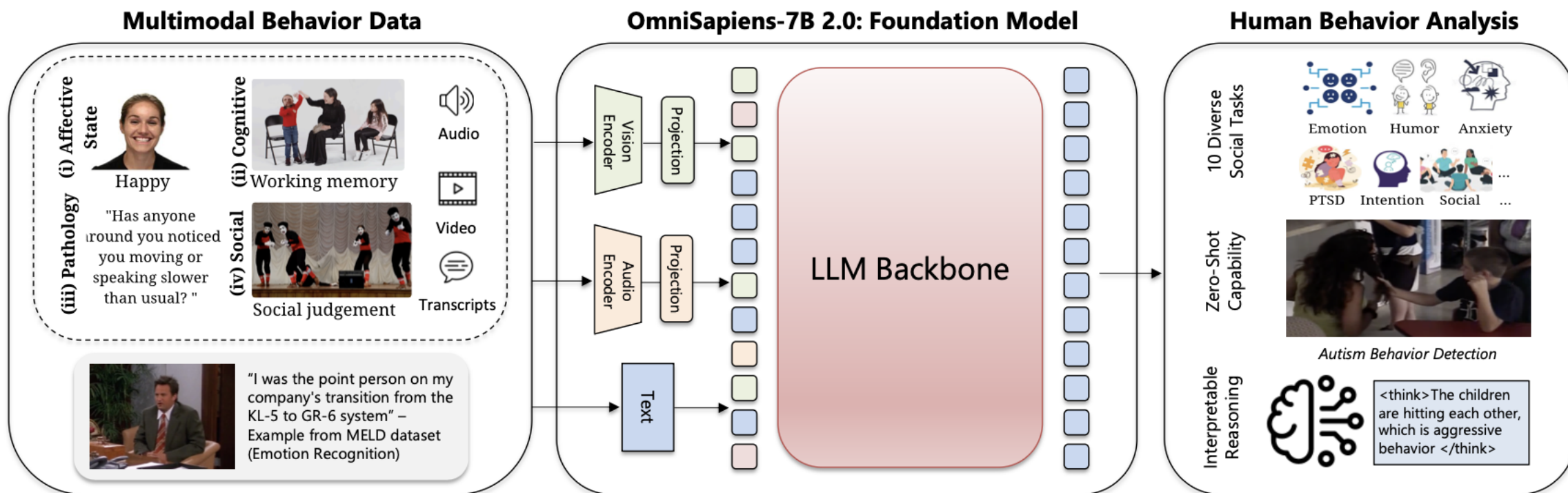


The architecture of MiniGPT-4.

Multimodal Context

https://github.com/MIT-MI/human_behavior_atlas

Interleaved tokens – language tokens, projected visual tokens, projected audio tokens etc.



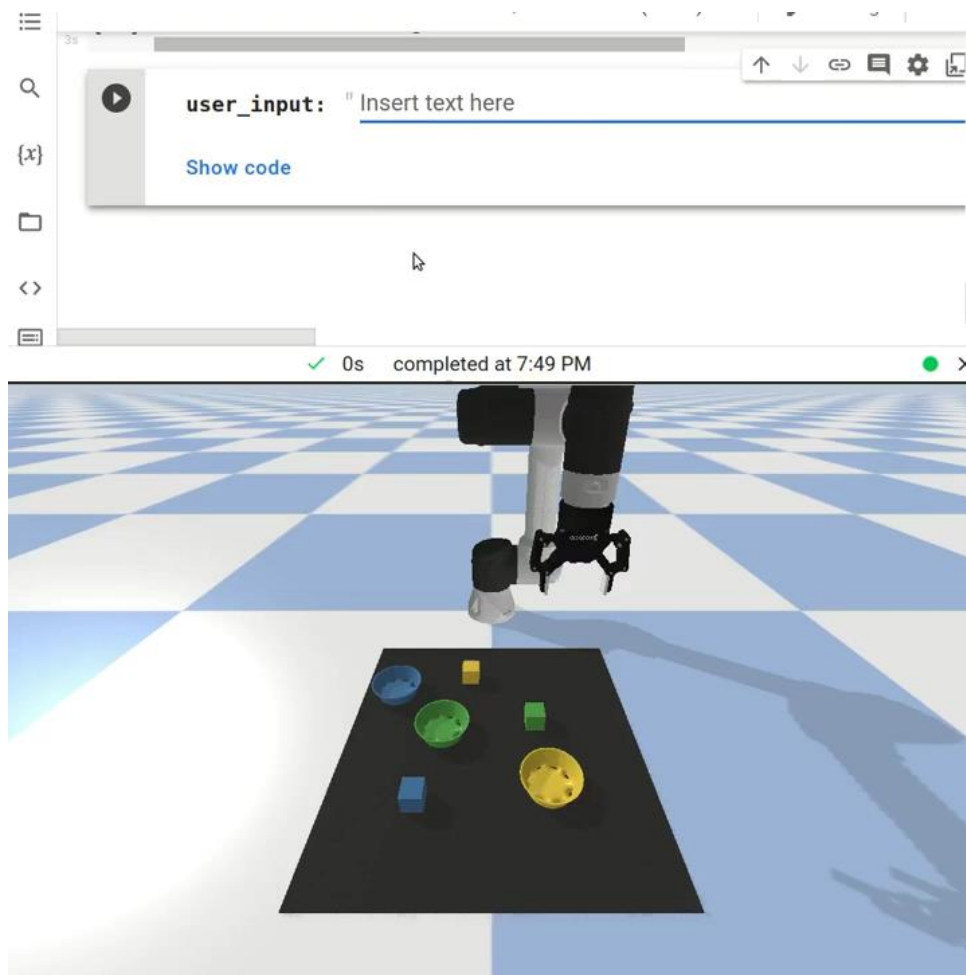
Multimodal Context

Image captioning

Zero-Shot
Socratic
Internet
Image
Captioning

Multimodal Context

Robot perception and planning



Multimodal Context

Video reasoning



11:09 AM: Places: **living room**. Objects: **remote control**, **television**, **netflix**. Commonsense activities: **watching netflix**. Most likely: **watching netflix**. I was **watching netflix**.

Language-based World-state History

8:31 AM: Places: **clean room**. Objects: **shorts**, **jeans**, **shirt**. Commonsense activities: **getting dressed**. Most likely: **getting dressed**. I was **getting dressed**.
 10:17 AM: Places: **kitchen**. Objects: **coffeemaker**, **waffle iron**, **kettle**. Commonsense activities: **making coffee**, **making waffles**. Most likely: **making coffee**. Summary: I was making coffee.



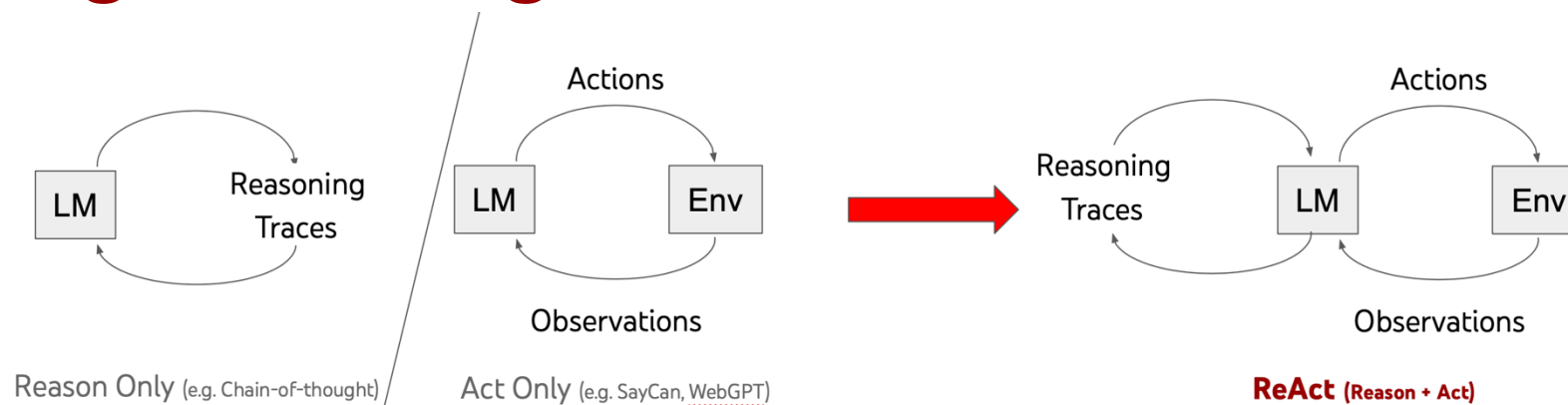
Contextual Reasoning Q&A

Q: Why did I go to the front porch today?

A: I went to the front porch today to receive a package.

Explanation: I saw on the porch a package and knew that I was expecting it.

Reasoning and Acting in Context



Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

Thought 1 I need to search Apple Remote and find the program it was designed for.

Act 1 Search[Apple Remote]

Obs 1 ... originally designed to control the Front Row media center program ...

Thought 2 Now I need to find what else can control Front Row.

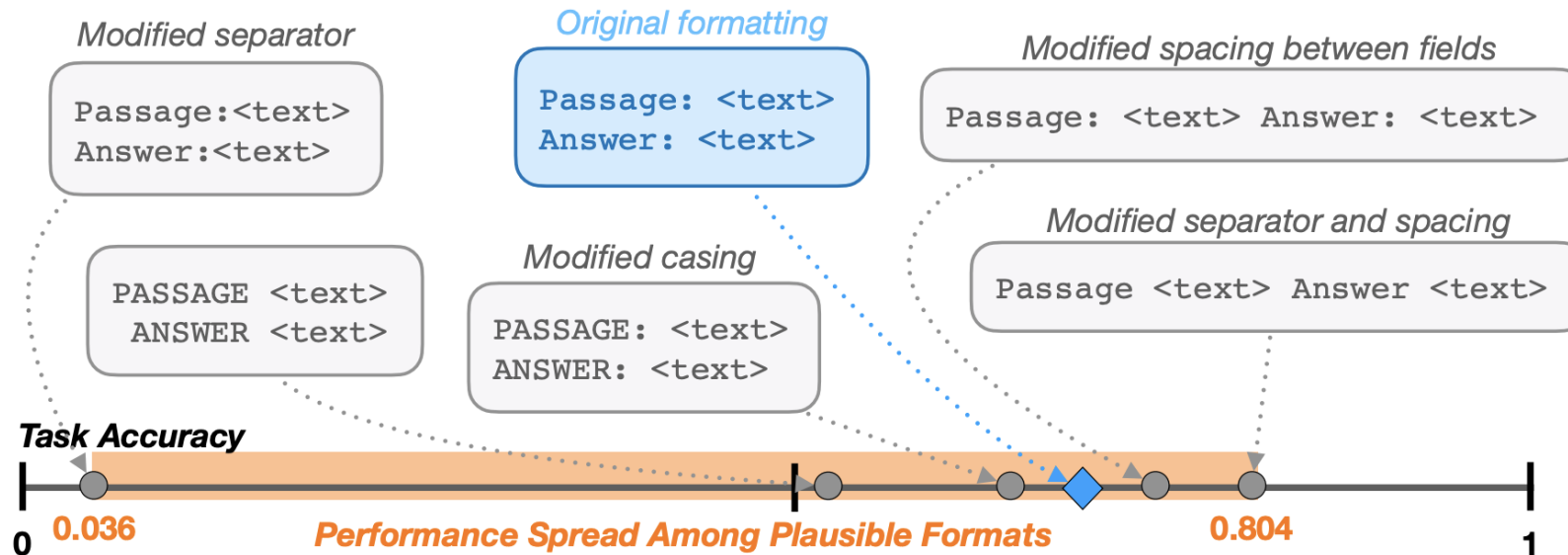
Act 2 Search[Front Row (software)]

Obs 2 Front Row ... controlled by an Apple Remote or the keyboard function keys.

Act 3 Finish[keyboard function keys] ✓

Prompt Sensitivity

Key finding: LLMs are (were?) sensitive to spurious changes to prompts.



Challenges in Prompting

Challenges in prompt design: confusions getting started, choosing the right instructions, expecting human capabilities, often had to seek help.

PROGRAMMING	Non-Experts' Intuitive Approach	Experts' Intuitive Approaches
Task requirement	Implicit	Explicit
Task specification	Implicit	Explicit
Code reuse	Unplanned	Planned
Code testing & verification	Overconfident	Cautious
Debugging	Opportunistic	Systematic
MACHINE LEARNING	Non-Experts' Intuitive Approach	Experts' Intuitive Approaches
ML task design	Directly map personal need to model task	Framing an achievable task
Debugging	Add more data, or “use deep learning”	Identify solvable underlying problems
Measuring success	Consider only one performance metric	Seek multiple performance indicators

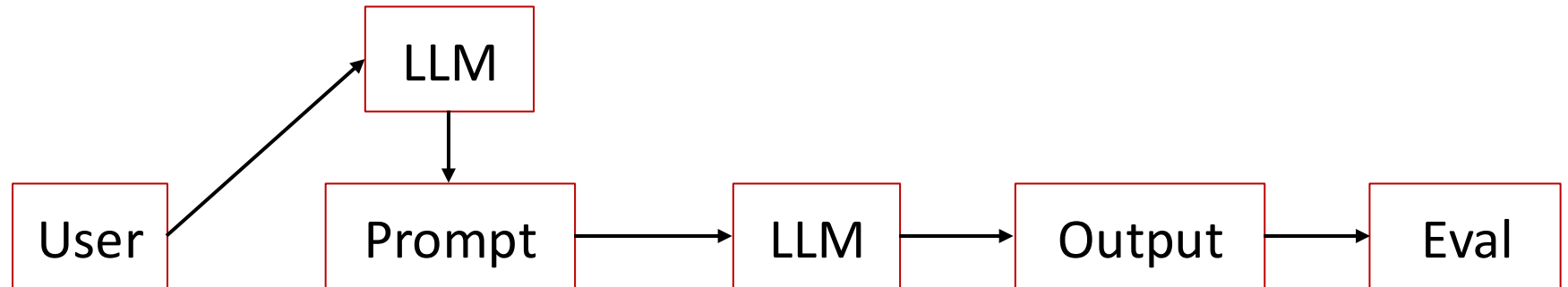
Table 2: Known differences in non-experts' and experts' approaches to software engineering [26] and to machine learning model building [50].

Prompt Optimization

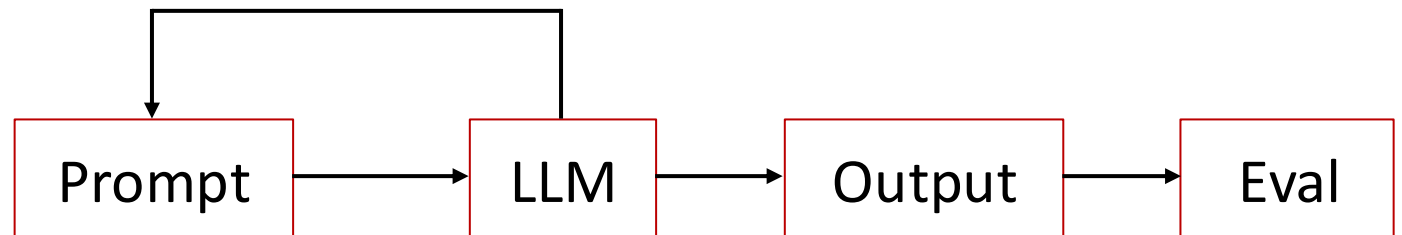
Standard
prompting



LLM-assisted
prompting



Prompt
optimization



Prompt Optimization

Main challenges of prompt evolution:

1. How to optimize over discrete non-differentiable tokens. Need techniques from discrete optimization.

→ Gradient-based or search-based.

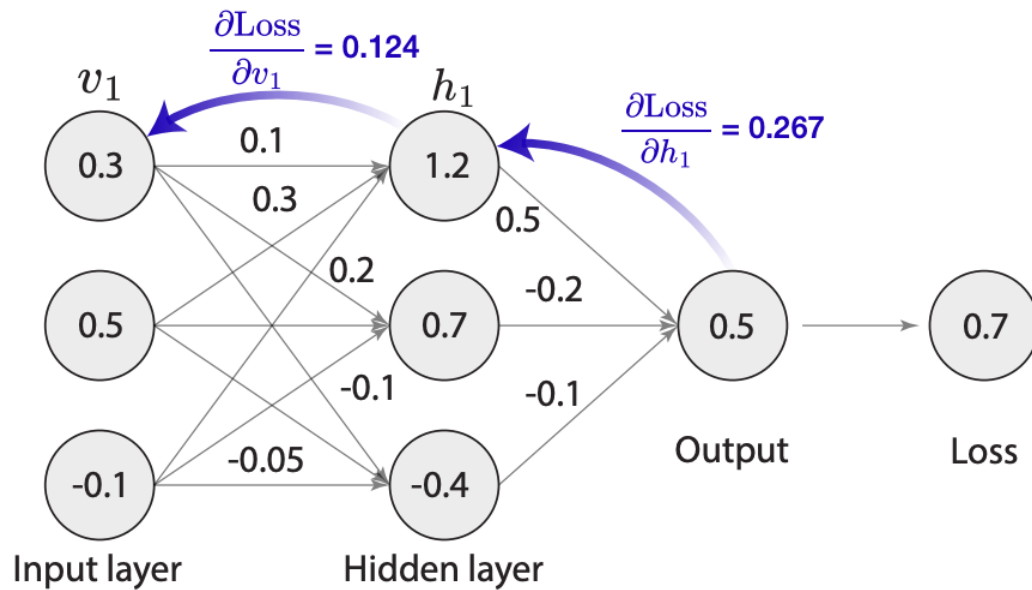
→ Gradients to update prompt directly vs update LLM that generates prompts.

2. Search space is extremely high-dimensional (space of all long strings). How to reduce the space through modular and composable prompts?

→ Text vs code modules, software engineering best practices.

Discrete Optimization

a Neural network and backpropagation using numerical gradients

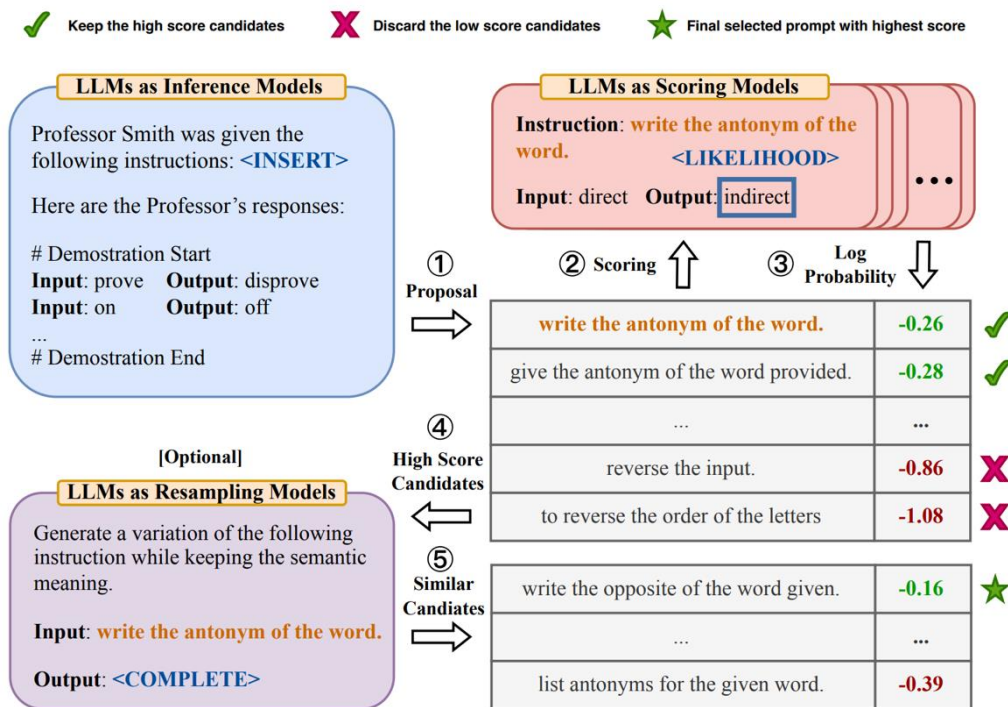


If inputs and layers are text, gradients are hard to compute.

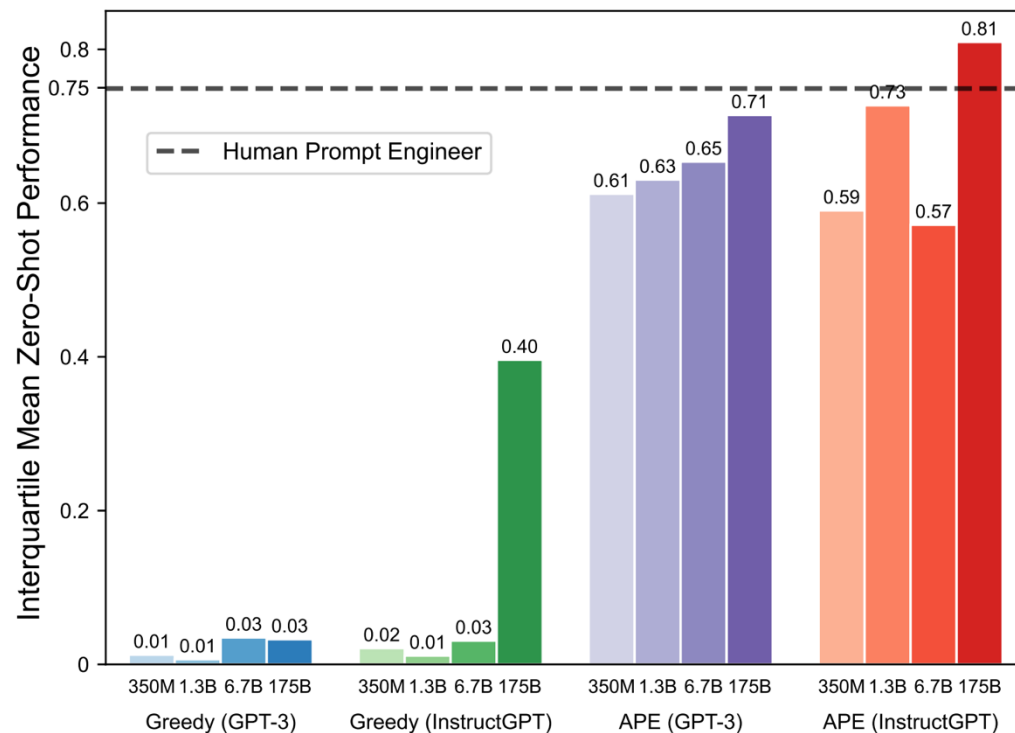
Take gradients wrt token embeddings?
But does not map to text.

Prompt Optimization via Text Generation

LLMs help generate and score prompts through black-box optimization/search



(a) Automatic Prompt Engineer (APE) workflow

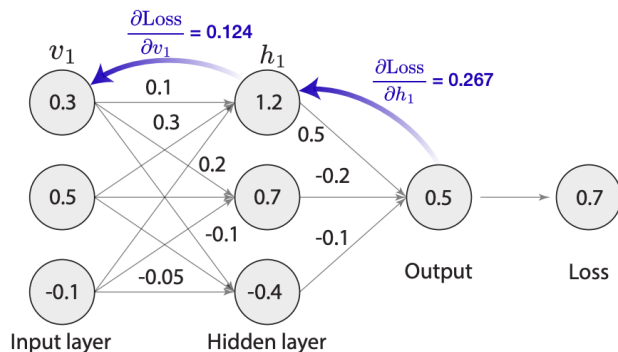


(b) Interquartile mean across 24 tasks

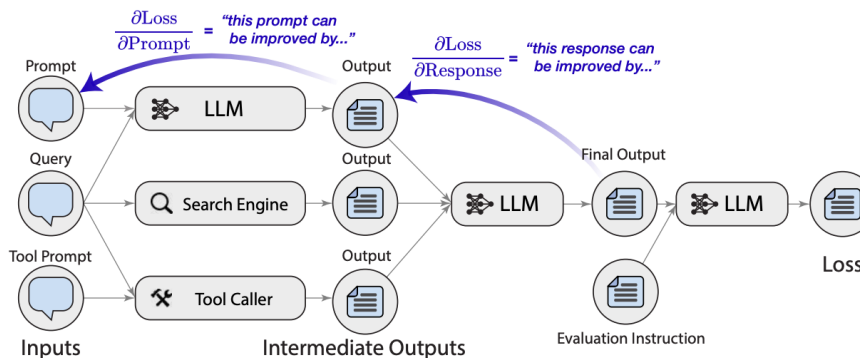
Prompt Optimization via Text Optimization: TextGrad

LLMs optimize prompts through gradient descent/first-order optimization

a Neural network and backpropagation using numerical gradients



b Blackbox AI systems and backpropagation using natural language 'gradients'



c 1 Analogy in abstractions

	Math	PyTorch	TextGrad
Input	x	<code>Tensor(image)</code>	<code>tg.Variable(article)</code>
Model	$\hat{y} = f_{\theta}(x)$	<code>ResNet50()</code>	<code>tg.BlackboxLLM("You are a summarizer.")</code>
Loss	$L(y, \hat{y}) = \sum y_i \log(\hat{y}_i)$	<code>CrossEntropyLoss()</code>	<code>tg.TextLoss("Rate the summary.")</code>
Optimizer	$\text{GD}(\theta, \frac{\partial L}{\partial \theta}) = \theta - \frac{\partial L}{\partial \theta}$	<code>SGD(list(model.parameters()))</code>	<code>tg.TGD(list(model.parameters()))</code>

2 Automatic differentiation

PyTorch and TextGrad share the same syntax for backpropagation and optimization.

Forward pass

`loss = loss_fn(model(input))`

Backward pass

`loss.backward()`

Updating variable

`optimizer.step()`

Prompt Optimization: TextGrad

$$\text{Prompt} + \text{Question} \xrightarrow{\text{LLM}} \text{Evaluation Instruction} + \text{Prediction} \xrightarrow{\text{LLM}} \text{Evaluation}, \quad (3)$$

Example gradient computation. In this example system, to improve the **Prompt** with respect to the evaluation, we instantiate an analog of the backpropagation [19] algorithm through the following:

Gradient computation for the simple graph: **Prompt** $\xrightarrow{\text{LLM}}$ Prediction $\xrightarrow{\text{LLM}}$ Evaluation

$$\frac{\partial \text{Evaluation}}{\partial \text{Prediction}} = \nabla_{\text{LLM}}(\text{Prediction}, \text{Evaluation}), \quad (4)$$

$$\frac{\partial \text{Evaluation}}{\partial \text{Prompt}} = \frac{\partial \text{Evaluation}}{\partial \text{Prediction}} \circ \frac{\partial \text{Prediction}}{\partial \text{Prompt}} = \nabla_{\text{LLM}}(\text{Prompt}, \text{Prediction}, \frac{\partial \text{Evaluation}}{\partial \text{Prediction}}), \quad (5)$$

Returns natural language feedback:
“This prediction can be improved by...”
to update the **Prompt**

How prediction changes
evaluation metric

1. Collect feedback to Prediction given Evaluation
2. Collect feedback to Prompt given (feedback to Prediction given Evaluation)

Prompt Optimization: TextGrad

Below is a flexible way to instantiate ∇_{LLM} to collect feedback for a simple system like $x \xrightarrow{\text{LLM}} y \xrightarrow{\text{LLM}} \mathcal{L}$:

Example implementation for the gradient operator

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x} = & \nabla_{\text{LLM}}(x, y, \frac{\partial \mathcal{L}}{\partial y}) \triangleq \text{"Here is a conversation with an LLM: \{x|y\}.\"} \\ & + \\ & \text{LLM}(\text{Here is a conversation with an LLM: \{x|y\}.} \\ & \quad \text{Below are the criticisms on \{y\}:} \\ & \quad \left\{ \frac{\partial \mathcal{L}}{\partial y} \right\} \\ & \quad \text{Explain how to improve \{x\}.),} \end{aligned} \tag{6}$$

Prompt Optimization: TextGrad

Example optimizer. In standard gradient descent, the current value of the variable is combined with the gradients through subtraction, e.g.:

$$\theta_{\text{new}} = \text{GradientDescent.step}(\theta, \frac{\partial \mathcal{L}}{\partial \theta}) = \theta - \frac{\partial \mathcal{L}}{\partial \theta}. \quad (7)$$

Continuing the gradient-based optimization analogy, we use *Textual Gradient Descent* (TGD):

Updating the **Prompt** variable in the simple graph via TGD.

$$\text{Prompt}_{\text{new}} = \text{TGD.step}(\text{Prompt}, \frac{\partial \text{Evaluation}}{\partial \text{Prompt}}). \quad (8)$$

to update the parameters, in this case, the **Prompt**. In particular, given the current variable and the gradients (feedback) we collected for this variable, the optimizer seeks to update this variable.

Prompt Optimization: TextGrad

A concrete way to instantiate TGD is the following:

Example implementation of one TGD iteration

$$x_{\text{new}} = \text{TGD.step}\left(x, \frac{\partial \mathcal{L}}{\partial x}\right) \triangleq \text{LLM}(\text{Below are the criticisms on } \{x\}: \quad (9)$$

$$\left\{ \frac{\partial \mathcal{L}}{\partial x} \right\}$$

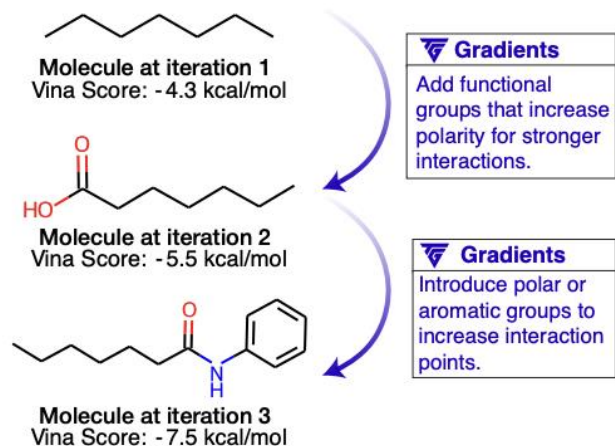
Incorporate the criticisms, and produce a new variable.).

where x is the variable we would like to improve, and $\frac{\partial \mathcal{L}}{\partial x}$ is the feedback we obtained for the variable during the backward pass^a. Similar to the gradient operator, this function also does not depend on the domain of application, and TGD implementation is the same across all uses of the framework.

^aThe exact prompts we use are different to ensure generality and flexibility; we use these examples only for exposition.

Prompt Optimization: TextGrad

d TextGrad for molecule optimization



e TextGrad for code optimization

```
for i in range(n):
    if nums[i] < k:
        balance -= 1
    elif nums[i] > k:
        balance += 1
    if nums[i] == k:
        result += count.get(balance, 0) +
        count.get(balance - 1, 0)
    else:
        result += count.get(balance, 0)
        count[balance] = count.get(balance, 0) + 1
```

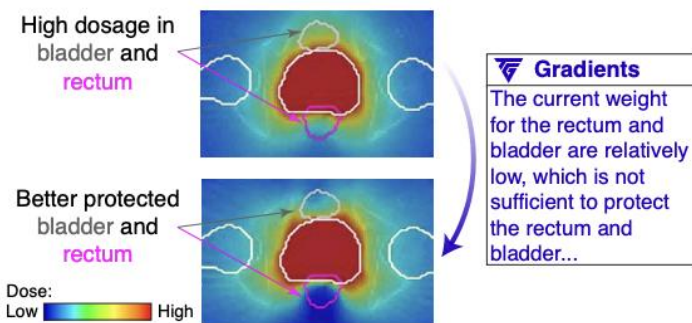
Code at iteration t

```
for i in range(n):
    if nums[i] < k:
        balance -= 1
    elif nums[i] > k:
        balance += 1
    else:
        found_k = True
    if nums[i] == k:
        result += count.get(balance, 0) +
        count.get(balance - 1, 0)
    else:
        count[balance] = count.get(balance, 0) + 1
```

Code at iteration t+1

Gradients
Handling `nums[i] == k`: The current logic does not correctly handle the case when `nums[i] == k`. The balance should be reset or adjusted differently when `k` is encountered. ...

f TextGrad for treatment plan optimization



g TextGrad for prompt optimization

You will answer a reasoning question. Think step by step. The last line of your response should be of the following format: 'Answer: \$VALUE' where VALUE is a numerical value.

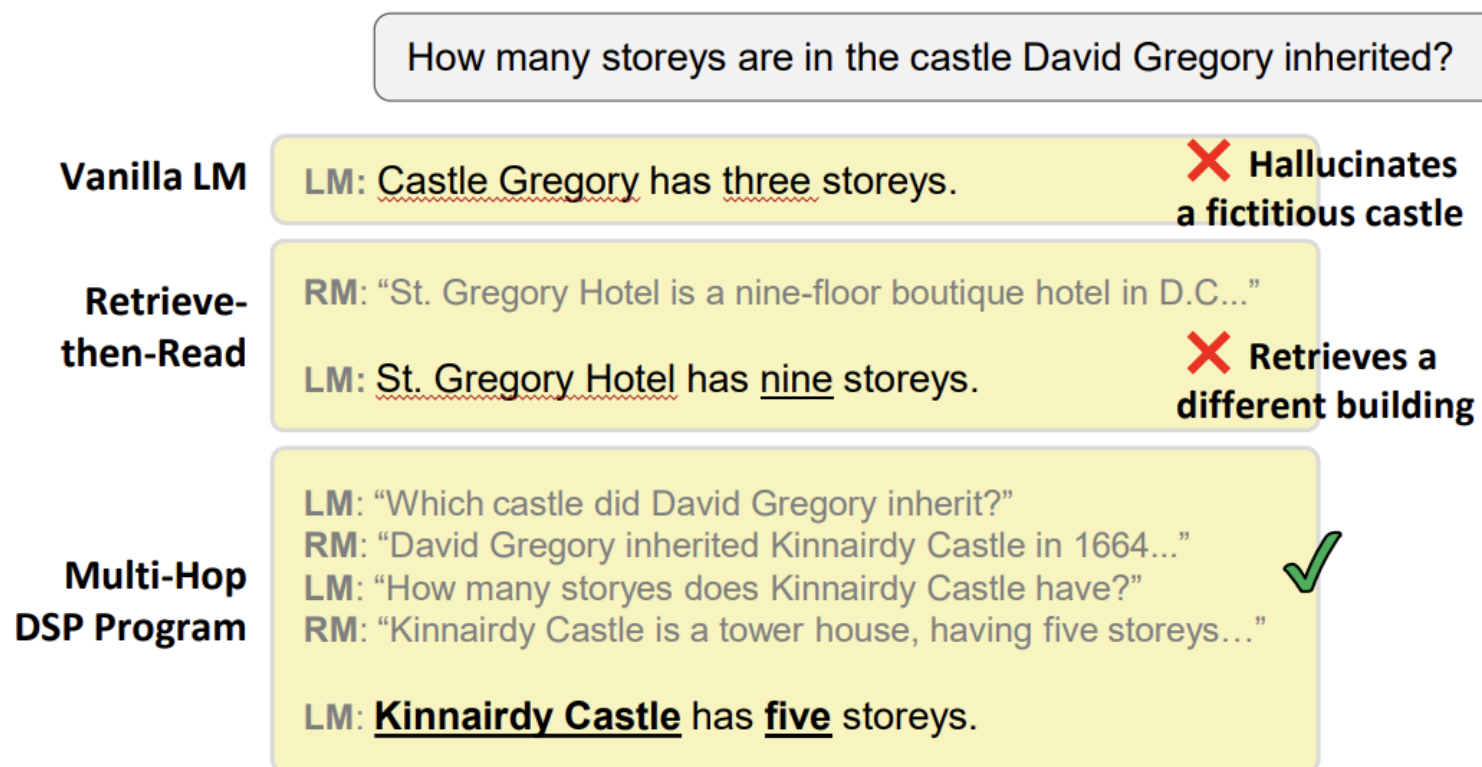
Prompt at initialization (Accuracy = 77.8%)

You will answer a reasoning question. List each item and its quantity in a clear and consistent format, such as '- Item: Quantity'. Sum the values directly from the list and provide a concise summation. Ensure the final answer is clearly indicated in the format: 'Answer: \$VALUE' where VALUE is a numerical value. Verify the relevance of each item to the context of the query and handle potential errors or ambiguities in the input. Double-check the final count to ensure accuracy."

Prompt after optimization (Accuracy = 91.9%)

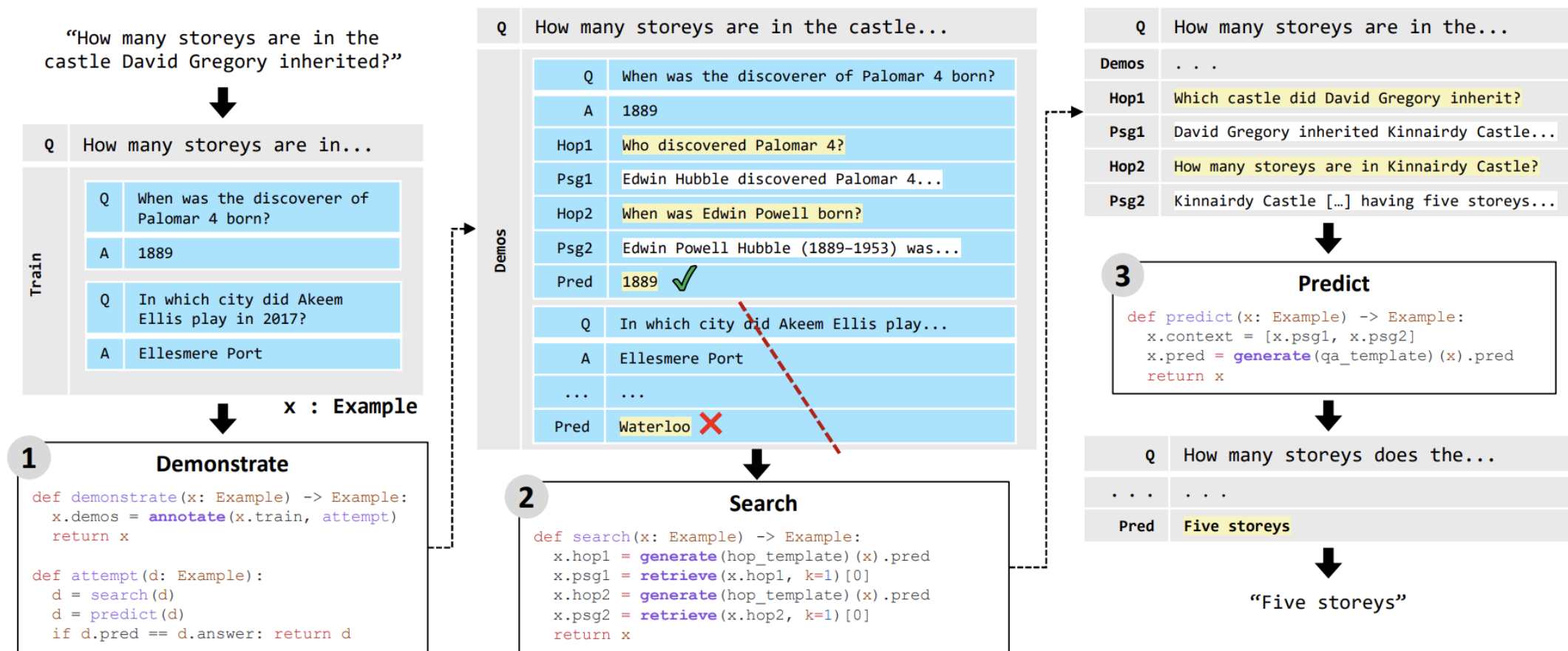
Prompt Optimization via Modular Programs

Problem: Optimization space is extremely high-dimensional (space of all long strings).
 Creating abstractions: each LLM step involves a set of common modules: retrieve, read etc.



DSPy Programming

DSPy pushes away from manipulating free-form strings and closer to programming (composing modular operators to build text transformation graphs). A compiler automatically generates optimized LM invocation strategies and prompts.



DSPy Programming

Example module instantiation in DSPy

```

1 class ChainOfThought(dspy.Module):
2     def __init__(self, signature):
3         # Modify signature from '*inputs -> *outputs' to '*inputs -> rationale, *outputs'.
4         rationale_field = dspy.OutputField(prefix="Reasoning: Let's think step by step.")
5         signature = dspy.Signature(signature).prepend_output_field(rationale_field)
6
7         # Declare a sub-module with the modified signature.
8         self.predict = dspy.Predict(signature)
9
10    def forward(self, **kwargs):
11        # Just forward the inputs to the sub-module.
12        return self.predict(**kwargs)

```

```

1 class RAG(dspy.Module):
2     def __init__(self, num_passages=3):
3         # 'Retrieve' will use the user's default retrieval settings unless overridden.
4         self.retrieve = dspy.Retrieve(k=num_passages)
5         # 'ChainOfThought' with signature that generates answers given retrieval & question.
6         self.generate_answer = dspy.ChainOfThought("context, question -> answer")
7
8     def forward(self, question):
9         context = self.retrieve(question).passages
10        return self.generate_answer(context=context, question=question)

```

DSPy Programming

Optimizing modules in DSPy

```
1 # Small training set with only questions and final answers.
2 qa_trainset = [dspy.Example(question="What is the capital of France?", answer="Paris")]
3
4 # The teleprompter will bootstrap missing labels: reasoning chains and retrieval contexts.
5 teleprompter = dspy.BootstrapFewShot(metric=dspy.evaluate.answer_exact_match)
6 compiled_rag = teleprompter.compile(RAG(), trainset=qa_trainset)
```

3 ways optimization is performed:

- 1: Candidate Generation. The compiler (recursively) finds all unique Predict modules (predictors) in a program and generates candidate values for its parameters: instructions, field descriptions, demonstrations (input–output pairs).
- 2: Parameter Optimization. Use hyperparameter tuning algorithms (e.g., random search or Bayesian optimization) to select among candidates: recall discrete optimization problem.
3. Higher-Order Program Optimization. Modify the control flow of the program e.g., ensemble to bootstrap multiple copies of the same program, running them in parallel and aggregating (e.g., majority voting).

Summary: Context Optimization

1. How to optimize over discrete non-differentiable tokens.

Need techniques from discrete optimization.

→ Gradient-based or search-based (TextGrad vs Automated Prompt Engineer)

→ Gradients to update prompt directly vs update LLM that generates prompts.

2. Search space is extremely high-dimensional (space of all long strings).

How to reduce the space through modular and composable prompts?

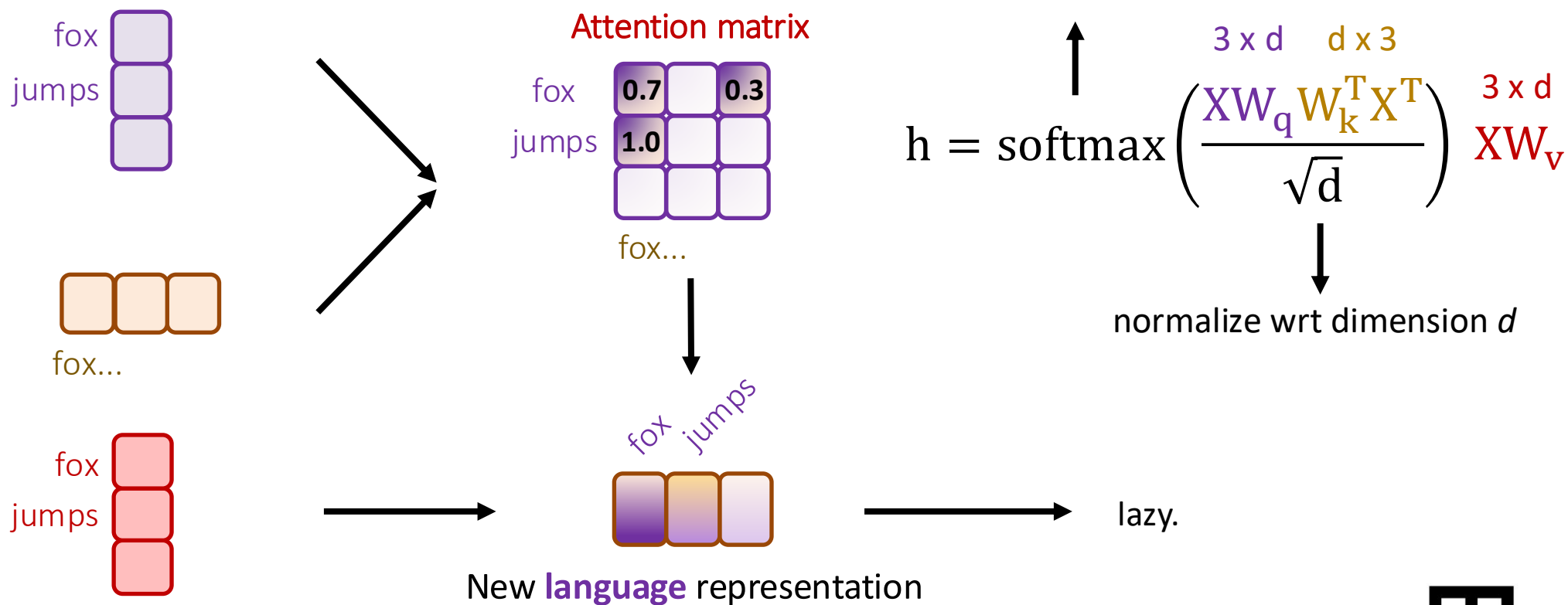
→ Text vs code modules, software engineering best practices (DPSy)

We will see more next week, in using LLMs to optimize and search over programs.

Long Context Models

Birth of attention-based models

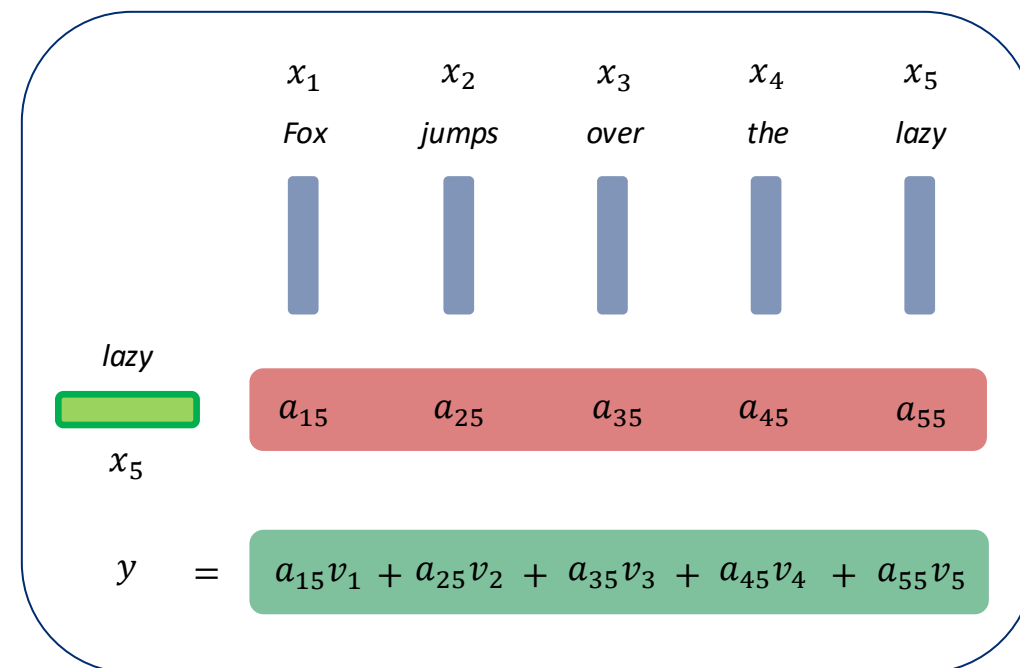
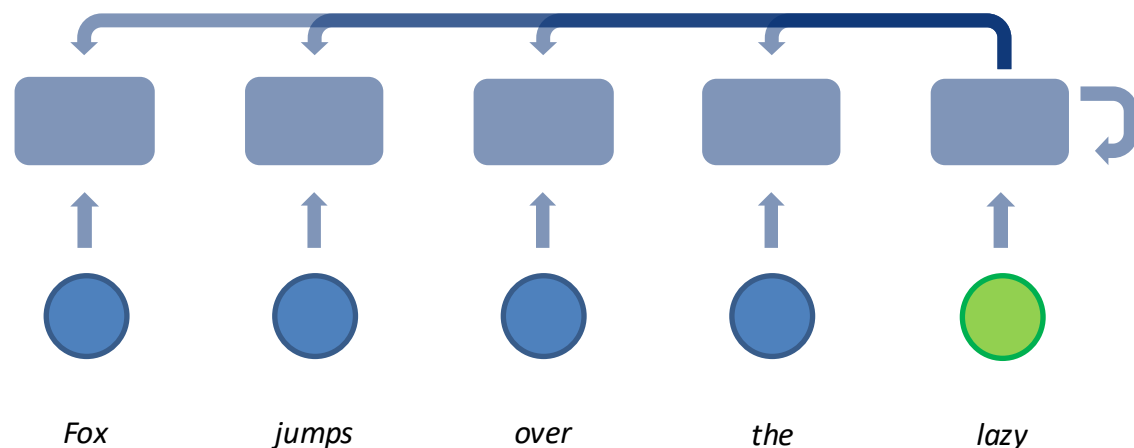
- Dynamic weights for different tokens



Transformer: A Recap

Birth of attention-based models

- Dynamic weights for different tokens



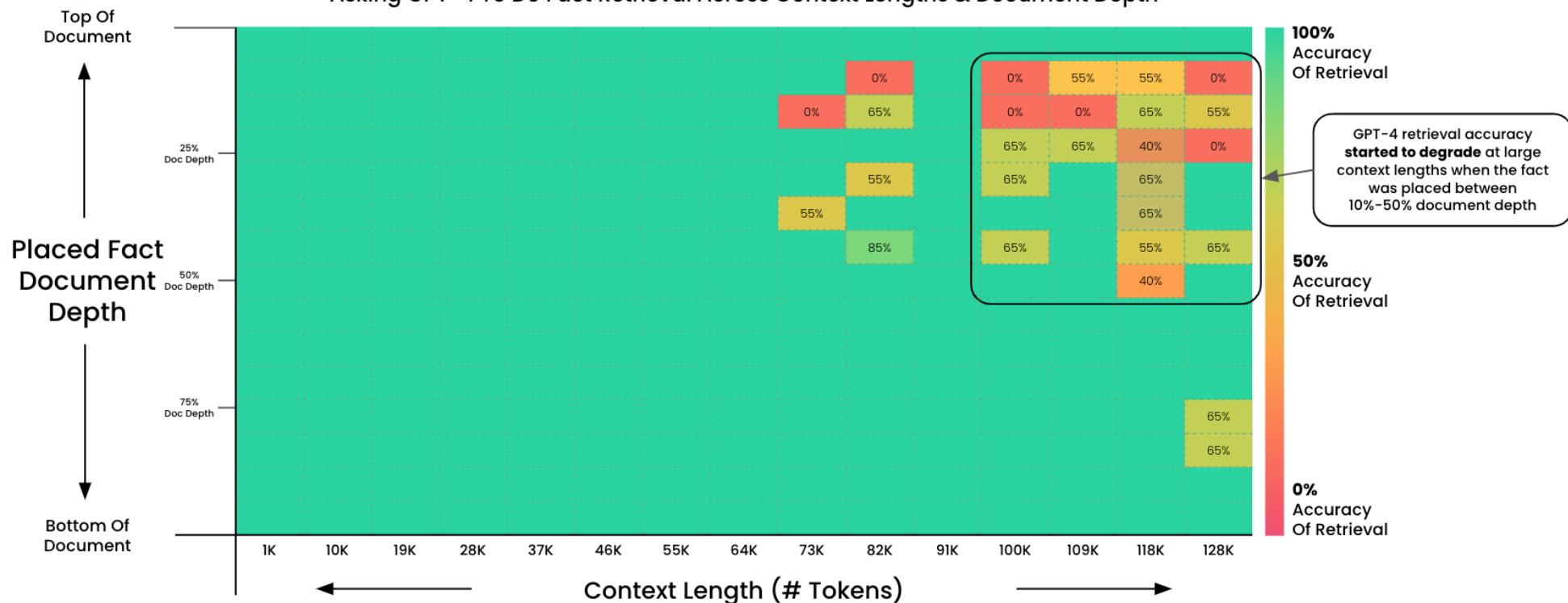
$$h = \text{softmax} \left(\frac{XW_q W_k^T X^T}{\sqrt{d}} \right) XW_v$$

$T \times d$ $d \times T$ $T \times d$
 X W_q W_k^T X^T X W_v

Long Context Models

Pressure Testing GPT-4 128K via "Needle In A HayStack"

Asking GPT-4 To Do Fact Retrieval Across Context Lengths & Document Depth



Goal: Test GPT-4 Ability To Retrieve Information From Large Context Windows

A fact was placed within a document. GPT-4 (1106-preview) was then asked to retrieve it. The output was evaluated for accuracy.

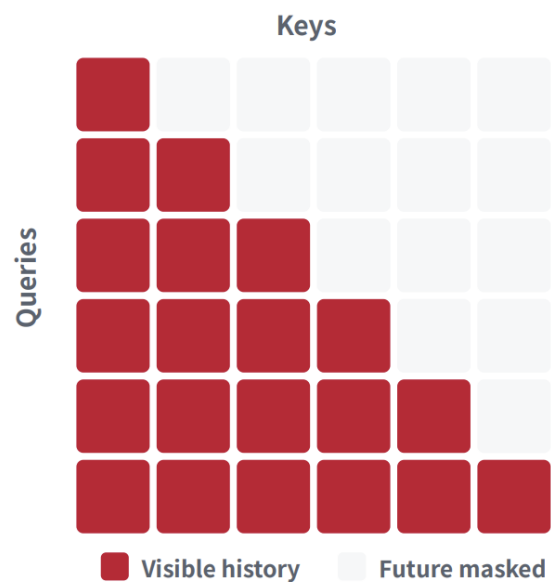
This test was run at 15 different document depths (top > bottom) and 15 different context lengths (1K > 128K tokens).

2x tests were run for larger contexts for a larger sample size.

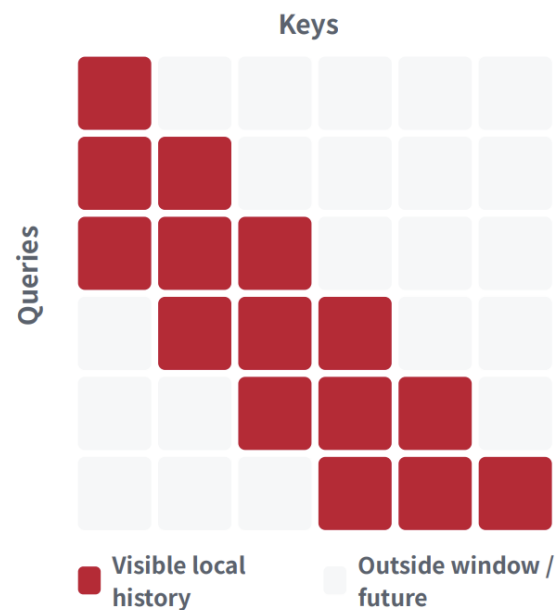
3 Ways to Deal with Long Context

1. New architectures: sliding window, sparse, linear attention etc.
 - Can improve model efficiency and reduces compute.
 - But needs re-design and re-training.
2. Long-context training: short-to-long training, synthetic data, extend position encodings
 - Helps expands the context window, simple to use.
 - But requires data and can be expensive.
3. Context management: Context summarization, memory systems, RAG
 - Improves the information pipeline, can scale.
 - Depends on retrieval and memory quality.

Long Context Models

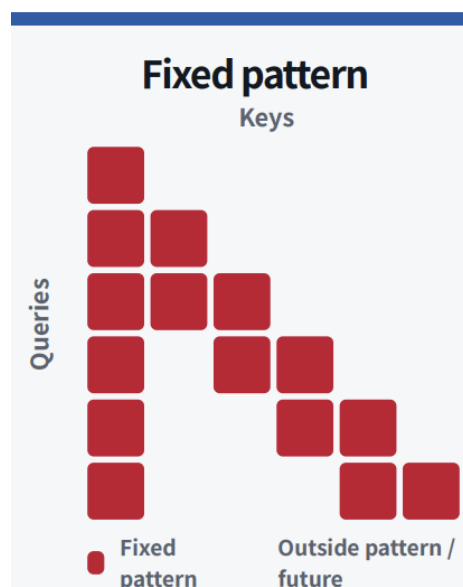


Full attention

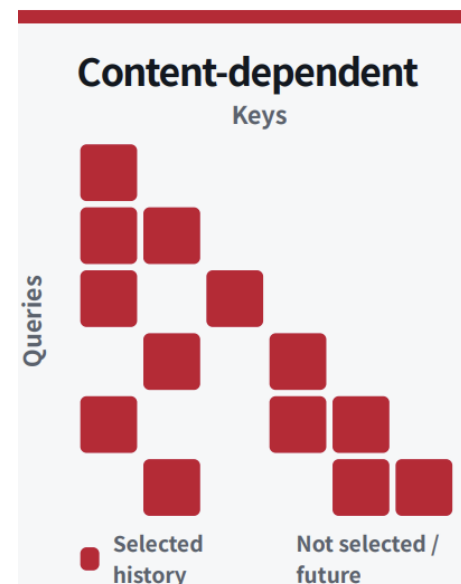


Sliding window
attention

Sparse Attention



Fixed global attention +
sliding window local attention

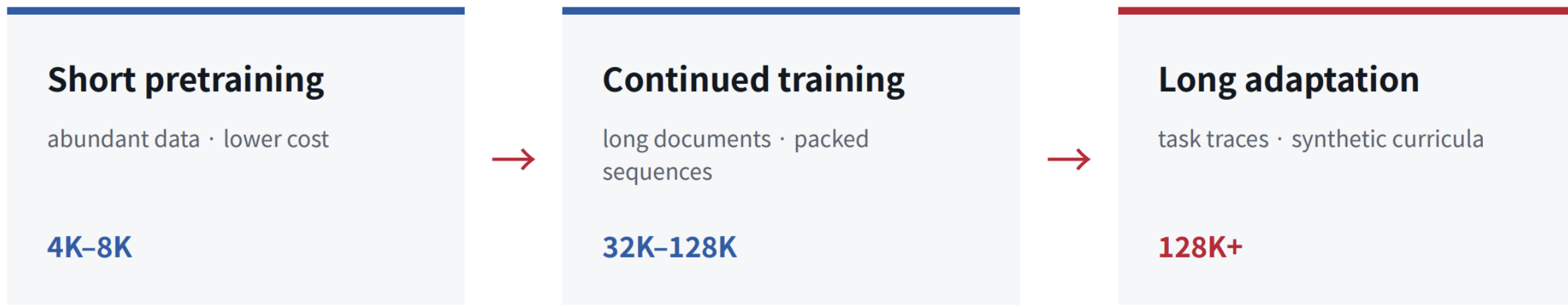


Context-dependent

Long Context Models Summary

Model	Local / stateful calculation	Global calculation	Local : global
Qwen3.8-Flash-Next	Gated DeltaNet	QSA sparse retrieval	$36 : 12 \approx 3:1$
GLM-5.3-Flash	Kimi Delta Attention	DSA + IndexPool	$34 : 11 \approx 3:1$
Kimi K3	Kimi Delta Attention	dense Gated MLA	$69 : 24 \approx 3:1$
Nemotron 3 Ultra	Mamba-2	dense GQA	$48 : 12 = 4:1$
Inkling-Small	512-token window	dense GQA	$35 : 7 = 5:1$
DeepSeek-V4-Pro-0813	128-token window branch	compressed sparse / dense	interleaved branches

Long Context Training



Examples of long-context data:

1. Long documents with coherent natural structure e.g., books, codebases, repositories, multiple documents.
2. Agent trajectories observations + actions + rewards.
3. Synthetic tasks with controlled evidence and distractors at designed positions.

Long Context Position Encodings

Pretraining range



Target range



evaluate target position t using the familiar phase t / s

Uniform position interpolation

$$R(t) \longrightarrow R(t/s) \quad \text{equivalently} \quad \theta_\ell \longrightarrow \theta_\ell/s$$

Long Context Models Summary

Model	Context-length training	Position method	How it reaches maximum context
Qwen3.8-Flash-Next	262K native; full curriculum undisclosed	partial RoPE	YaRN extension to 1M
DeepSeek V4	4K → 16K → 64K → 1M	partial RoPE	progressive training + YaRN
GLM-5.3-Flash	1M native; exact curriculum undisclosed	NoPE in main attention	no RoPE rescaling
Kimi K3	8K → 64K → 256K → 1M	NoPE	progressive training to 1M
Nemotron 3 Ultra	1M long-context CPT; SFT to 512K	implicit order in Mamba; no RoPE	trained/evaluated at long lengths
Inkling	1M context; curriculum undisclosed	relative position embedding	learned relative representation

Long Context Evaluation

Category	Dataset	Metrics	Description
Retrieval-augmented generation	Natural Questions	SubEM	Factoid question answering
	TriviaQA	SubEM	Trivia question answering
	PopQA	SubEM	Long-tail entity question answering
	HotpotQA	SubEM	Multi-hop question answering
Generation with citations	ALCE ASQA	Recall, Cite	Answer ambiguous questions with citations
	ALCE Qampari	Recall, Cite	Answer factoid questions with citations
Passage re-ranking	MS MARCO	NDCG@10	Rerank passage for a query
Many-shot in-context learning	TREC Coarse	Accuracy	Question type classification, 6 labels
	TREC Fine	Accuracy	Question type classification, 50 labels
	NLU	Accuracy	Task intent classification, 68 labels
	BANKING77	Accuracy	Banking intent classification, 77 labels
	CLINC150	Accuracy	Intent classification, 151 labels
Long-document QA	NarrativeQA	Model-based	Book and movie script QA
	∞ BENCH QA	ROUGE F1	Novel QA with entity replacement
	∞ BENCH MC	Accuracy	Novel multiple-choice QA with entity replacement
Summarization	∞ BENCH Sum	Model-based	Novel summarization with entity replacement
	Multi-LexSum	Model-based	Summarizing multiple legal documents
Synthetic recall	JSON KV	SubEM	Retrieve a key in JSON dictionary
	RULER MK Needle	SubEM	Retrieve the needle (a number) within noisy needles
	RULER MK UUID	SubEM	Retrieve the needle (a UUID) within noisy needles
	RULER MV	SubEM	Retrieve multiple values for one needle (key)

Lecture Summary

- Types of context: text and multimodal, prompts, memory
- Prompt optimization
- Long-context models
- Context management and memory
- Guest speaker: agentic context engineering
- Demo of context evolution